

STM32 con CubeIDE, VS Code y Wokwi

Ingeniería de Software de Vuelo para Sistemas Espaciales Críticos · 2.º cuatrimestre 2026

Versión 0.4 · 29/09/2026 · probada con STM32CubeIDE 1.18.1, VS Code 1.139 y Wokwi for VS Code 3.7, en Windows 11

Para quién es. Para vos, que ya instalaste STM32CubeIDE en la clase de la semana 4 y tenés que hacer el TP2 simulado en Wokwi. El método es **el de la cátedra**, juntado y ordenado en un solo lugar; lo que agregamos nosotros va aparte, marcado, y se puede saltar. No explica C ni la HAL, y no resuelve el TP: te deja el entorno andando para que lo resuelvas vos.

Los colores no son adorno: cada uno dice una cosa y sólo una.

Azul	en el texto, La idea : — lo que no hay que perder.
Rojo	en el texto, Ojo : — donde se traba casi todo el mundo.
Violeta	lo que pide la cátedra, sacado de sus clases y de sus TP.
Verde azulado	una mejora : algo que agregamos nosotros y no es de la cátedra. Se puede saltar.
Rosa	la posta: lo mismo, dicho en criollo y sin vueltas.

Índice

1	Qué hace cada herramienta	1
2	Lo que tenés que tener instalado	1
3	El TP2, paso a paso	1
3.1	En CubeIDE: proyecto, pines y código	1
3.2	En VS Code: la simulación	2
4	Dónde se escribe el código	3
5	Si algo no anda	4
6	Después: la placa real (TP3)	4
7	Los archivos, completos	5

SECCIÓN 1

Qué hace cada herramienta

En la materia se trabaja con tres programas, y cada uno tiene un papel distinto. Confundirlos es la fuente de la mitad de los problemas, así que conviene tenerlo claro antes de tocar nada.

Tarea	Dónde	Cómo
Configurar pines y periféricos	CubeIDE	el archivo <code>.ioc</code> → Generate Code
Escribir el código	CubeIDE	en <code>main.c</code> , sólo entre los <code>USER CODE BEGIN/END</code>
Compilar	CubeIDE	el martillo o <code>Ctrl+B</code> → Build Finished , 0 errores
Simular	VS Code + Wokwi	<code>wokwi.toml + diagram.json</code> → Wokwi: Start Simulator
Grabar la placa real	CubeIDE	Run , con la placa enchufada (TP3, sección 6)

La idea: todo termina en un archivo `.elf`, que es el programa ya compilado. CubeIDE lo fabrica; Wokwi lo carga en una placa simulada; y en el TP3 el mismo CubeIDE lo graba en la placa de verdad. Si tenés claro dónde queda el `.elf` y quién lo usa, tenés claro el flujo entero.

LO QUE PIDE LA CÁTEDRA — POR QUÉ SE SIMULA OTRA PLACA

La placa de la materia es la **NUCLEO-F446RE**, pero Wokwi no simula sus periféricos. Sí simula la **NUCLEO-C031C6**, que se programa con el mismo patrón de código HAL. Por eso el TP2 se hace sobre la C031C6, y el TP3 lleva ese mismo código a la F446RE real.

SECCIÓN 2

Lo que tenés que tener instalado

1. **STM32CubeIDE 1.18.1**, la versión que fijó la cátedra, en la ruta que propone el instalador (`C:\ST\STM32CubeIDE_1.18.1\`). Al instalar, los drivers de ST-LINK van **todos** tildados, aunque todavía no tengás la placa: es la única oportunidad fácil de instalarlos.
2. **VS Code** y, adentro, la extensión **Wokwi for VS Code** (Extensiones → buscás «Wokwi» → la que se llama así).
3. La **licencia gratuita de Wokwi**: `Ctrl+Shift+P` → **Wokwi: Request a New License** → en el navegador, «GET YOUR LICENSE» → volvé a VS Code. Si no la toma sola, **Wokwi: Manually Enter License Key** y pegás la clave.

Ojo: la licencia gratuita **dura 30 días**; lo dice la misma página de Wokwi al darla. Cuando vence, pedís otra con el mismo comando y listo: no hay que reinstalar nada.

SECCIÓN 3

El TP2, paso a paso

3.1 En CubeIDE: proyecto, pines y código

1. `File` → `New` → `STM32 Project` → pestaña **Board Selector** → buscás **NUCLEO-C031C6** (no la F446RE) → le ponés nombre → **Finish**. Conviene un nombre **sin espacios ni acentos**: la cátedra lo pide para el workspace, y ese nombre termina en la ruta del `.elf`.
2. Después del **Finish** aparece **Board Project Options**: los componentes de la placa (LED verde LD4, botón de usuario y **Virtual Com Port**, los tres tildados) y «Generate demonstration code», sin tildar. Se deja así y **OK**. El Virtual Com Port es el que después da la consola de `printf`.

3. A la pregunta «Initialize all peripherals with their default Mode?», **Yes**. Con eso el LED de la placa queda en PA5 y se genera sola la consola por el ST-LINK (BSP_COM_Init), que es la que usa `printf`.
4. En el `.ioc`, vista **Pinout & Configuration**: clic en el pin → `GPIO_Output` para cada salida; para el ADC, `Analog → ADC1 → IN0` (queda en PA0, con 12 bits). PA2 y PA3 aparecen en fucsia: son la consola del ST-LINK, vienen reservadas de fábrica y no se tocan.
5. Guardás y apretás **Generate Code** (el engranaje, o `Project → Generate Code`).
6. Abrís `Core/Inc/stm32c0xx_nucleo_conf.h` y confirmás que diga `#define USE_COM_LOG 1U`. Con eso `printf` sale por la consola sin escribir nada más.
7. Escribís tu código en `main.c`, en los bloques de la sección 4, y compilás con el martillo. La **Console** tiene que terminar en `Build Finished` con 0 errores, y el `.elf` queda en la carpeta `Debug` del proyecto, con el nombre del proyecto.

3.2 En VS Code: la simulación

Wokwi **no es un programa aparte**: es una extensión de VS Code, y la simulación se abre como una pestaña más. Lo que simula lo leen dos archivos de texto, y todo lo demás es cómo se arman.

Una sola vez:

1. Extensiones (`Ctrl+Shift+X`) → buscás **Wokwi Simulator** → **Install**.
2. `Ctrl+Shift+P` → `Wokwi: Request a New License` → se abre el navegador, entrás con tu cuenta y vuelve solo a VS Code. La licencia dura 30 días; cuando vence, lo mismo.

Por proyecto, paso a paso:

1. **Dónde van los archivos**. En la carpeta que abrís en VS Code (`File → Open Folder`), en la raíz: la extensión busca `wokwi.toml` ahí y en ningún otro lado. La cátedra abre la carpeta del workspace (`...\workspace_1.18.1\`), así que van ahí; más abajo hay una variante más cómoda.
2. **De dónde salen**. Hay tres formas, de la más rápida a la más a mano: (a) el script `preparar-vscode.ps1` los escribe solo (ver la mejora de abajo); (b) copiás los de la sección 7 y cambiás el nombre del proyecto; (c) los creás vacíos: `wokwi.toml` con cuatro líneas, `diagram.json` desde wokwi.com (ver el paso de editar).
3. **wokwi.toml: qué programa carga**. Cuatro líneas, con la ruta relativa a la carpeta abierta y barras `/`:

```
[wokwi]
version = 1
firmware = 'tp2/Debug/tp2.elf'
elf = 'tp2/Debug/tp2.elf'
```

Si la ruta está mal, Wokwi no arranca: probala con `Ctrl+clic` sobre ella.

4. **diagram.json: qué circuito simula**. Tiene dos listas: `parts` (la placa `board-st-nucleo-c031c6` y cada componente, con un `id`) y `connections` (cada cable: `["nucleo:PA5", "led1:A", "green", []]`, de un pin a otro).
5. **Editarlo — tres formas**:
 - **visual, en wokwi.com**, como el profe, y es **la única visual gratis**: proyecto nuevo con la misma placa, armás el circuito (+ agrega componentes, un cable se tira con clic en un pin y clic en otro), pestaña `diagram.json`, copiás todo y lo pegás en tu archivo;
 - **visual, en VS Code**: abrir `diagram.json` muestra el circuito, pero **editarlo ahí pide un plan pago** (Hobby+ o Pro): con la licencia gratis sale «Upgrade to Edit Diagram». **Close**, y se edita de otra forma;
 - **a mano, como texto**: para cambiar un pin o agregar un cable es lo más rápido. Cada `id` de `connections` tiene que existir en `parts`.
6. **Correrlo**. Compilás primero (el `.elf` tiene que existir), después `Ctrl+Shift+P → Wokwi: Start Simulator`. Se abre la placa simulada en una pestaña; lo que imprime `printf` sale en la terminal de VS Code, y los botones del circuito se aprietan con el mouse.
7. **Cortar o reiniciar**. El botón de arriba a la izquierda de la pestaña de la simulación; `Wokwi: Stop Simulator` la cierra.

Ojo: la pestaña de la simulación tiene que quedar **a la vista**: si la tapás con otra o minimizás VS Code, Wokwi pausa la simulación y parece colgada.

Qué dice cada archivo, en detalle:

1. En esa carpeta van dos archivos. `wokwi.toml` dice **qué programa** cargar: `firmware` y `elf` apuntan al `.elf`, con la ruta relativa a la carpeta abierta. `diagram.json` dice **qué circuito** simular: la placa `board-st-nucleo-c031c6`, los componentes y los cables. Los dos, completos, están en la sección 7.
2. El circuito se arma con un **editor visual**: el de VS Code, que se abre solo al abrir `diagram.json`, o el de `wokwi.com`, como en la clase, copiando después el `diagram.json` que genera. Al arrastrar un cable, el editor te muestra qué pines hay libres.
3. El monitor serie queda conectado si `diagram.json` une `$serialMonitor` con `PA2` y `PA3`, cruzados: el TX de uno va al RX del otro.
4. `Ctrl+Shift+P` → `Wokwi: Start Simulator`. Se abre la placa simulada, y lo que imprime `printf` aparece en la terminal de VS Code.

La idea: el ciclo de todos los días es **cambiar el código** → **compilar** → **reiniciar la simulación**, con el botón verde de arriba a la izquierda del simulador. Si te salteás un paso, Wokwi sigue corriendo el `.elf` anterior.

LA POSTA

CubeIDE es el taller: ahí armás y compilás. Wokwi es el banco de prueba: ahí enchufás lo que salió del taller. Cada vez que cambiás algo en el taller, lo tenés que volver a llevar al banco —compilar y reiniciar—, o te vas a pasar diez minutos probando la versión de hace diez minutos.

MEJORA, NO ES DE LA CÁTEDRA — UN CIRCUITO POR PROYECTO

Con `wokwi.toml` y `diagram.json` en la raíz del workspace hay un solo circuito, y cada vez que cambiás de proyecto tenés que editar la ruta. Si los ponés **adentro de la carpeta de cada proyecto** —con `firmware = 'Debug/<proyecto>.elf'`— y abrís en VS Code la carpeta **del proyecto**, cada ejercicio conserva su circuito y no tocás ninguna ruta. Si preferís dejarlos en la raíz, la extensión trae `Wokwi: Select Config File` para elegir cuál usar.

MEJORA, NO ES DE LA CÁTEDRA — COMPILAR DESDE VS CODE

Para no ir y venir entre ventanas: `Ctrl+Shift+B` en VS Code corre el **mismo** `make` y el **mismo** `arm-none-eabi-gcc` que usa CubeIDE, sobre el `makefile` que CubeIDE genera, y deja el mismo `.elf`; los errores salen en **Problems** con archivo y línea. Lo deja armado el script `preparar-vscode.ps1`, que acompaña esta guía: saca cada dato del proyecto y escribe la tarea de compilar, la configuración para que VS Code reconozca la HAL y los dos archivos de Wokwi (estos últimos, sólo si no existen). Se corre una vez por proyecto:

```
cd "<carpeta donde bajaste preparar-vscode.ps1>"
powershell -ExecutionPolicy Bypass -File preparar-vscode.ps1 -Proyecto "<ruta de tp2>"
```

La condición: el proyecto tiene que haberse compilado **una vez en CubeIDE**, y cada vez que tocás el `.ioc` volvé a compilar ahí, porque el `makefile` lo escribe él. Probado sobre el proyecto de la clase, en una ruta con espacios.

SECCIÓN 4

Dónde se escribe el código

Generate Code reescribe `main.c` entero y sólo respeta lo que está entre un par de comentarios `USER CODE BEGIN / USER CODE END`. Lo que quede afuera se pierde, y sin aviso.

Bloque de <code>main.c</code>	Qué va
USER CODE BEGIN Includes	<code>#include <stdio.h></code> y otros
USER CODE BEGIN PV	variables globales
USER CODE BEGIN 0	funciones propias (por ejemplo, una que lea el ADC)
USER CODE BEGIN 2	lo que corre una sola vez antes del lazo (por ejemplo, calibrar el ADC)
USER CODE BEGIN WHILE	el cuerpo del <code>while (1)</code> , como el Blink de la clase

Ojo: `USE_COM_LOG` vive en un archivo que CubeMX **puede regenerar entero** si volvéis a tocar el `.ioc`. Revisalo después de cada **Generate Code**: si se pisó, `printf` deja de salir y el código no tiene nada de malo.

SECCIÓN 5

Si algo no anda

Síntoma	Qué hacer
Wokwi no arranca o no encuentra el firmware	casi siempre, la ruta de <code>wokwi.toml</code> no apunta a un <code>.elf</code> que exista. Probala con <code>Ctrl+click</code> ; si no está, compilá
Wokwi pide licencia	venció (dura 30 días): Wokwi: Request a New License
<code>printf</code> no muestra nada	en este orden: <code>USE_COM_LOG</code> en 1U; el monitor en PA2/PA3 en <code>diagram.json</code> ; la pestaña del simulador a la vista (tapada o minimizada, Wokwi pausa la simulación)
el cambio no se ve en la simulación	no recompilaste, o no reiniciaste la simulación
un pin sale naranja en el <code>.ioc</code>	conflicto de configuración: ese pin o su periférico ya está en uso
<code>Ctrl+Shift+B</code> no encuentra <code>Debug</code> o el <code>makefile</code> (con la mejora)	el proyecto nunca se compiló en CubeIDE: un martillo y listo
<code>undefined reference to ...</code> después de tocar el <code>.ioc</code> (con la mejora)	el <code>makefile</code> quedó viejo: un martillo en CubeIDE

SECCIÓN 6

Después: la placa real (TP3)

LO QUE PIDE LA CÁTEDRA — TP3, DE LA SIMULACIÓN A LA NUCLEO-F446RE

La placa se conecta por USB en el conector **CN1**, el del ST-LINK. Ya no alcanza con compilar: **Run** compila y además graba el micro. La primera vez pide una **Launch Configuration**: elegís `STM32 Cortex-M C/C++ Application` y aceptás lo que viene por defecto. Para `printf`, el TP3 usa el patrón directo: escribís a mano `__io_putchar` en `USER CODE BEGIN 4`, sobre `huart2`, con `#include <stdio.h>`, y abrís una terminal serie a 115200 baudios.

MEJORA, NO ES DE LA CÁTEDRA — EL TP3 EN WINDOWS

El TP3 da los comandos de Linux (/dev/ttyACM0, picocom). En Windows, el mismo puerto aparece como COMx en el **Administrador de dispositivos** → **Puertos (COM y LPT)**, y te sirve cualquier terminal serie a 115200 baudios. Esta parte todavía no se probó con la placa en la mano.

SECCIÓN 7

Los archivos, completos

diagram.json — el de la clase: la placa, un LED en PB2 y el monitor serie. Para el TP2 agregás componentes (wokwi-led, wokwi-potentiometer) en wokwi.com o como texto: una línea en **parts** por componente y una en **connections** por cable.

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    { "type": "board-st-nucleo-c031c6", "id": "nucleo", "top": 10.43, "left": -164.18,
    "attrs": {} },
    { "type": "wokwi-led", "id": "led1", "top": 25.2, "left": 138.2, "attrs": { "color":
    "green" } }
  ],
  "connections": [
    [ "$serialMonitor:TX", "nucleo:PA3", "", [] ],
    [ "$serialMonitor:RX", "nucleo:PA2", "", [] ],
    [ "nucleo:GND.9", "led1:C", "black", [ "h-21.55", "v-96", "h115.2" ] ],
    [ "nucleo:PB2", "led1:A", "green", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

wokwi.toml — en la raíz del workspace, como en la clase; la ruta incluye la carpeta del proyecto:

```
[wokwi]
version = 1
firmware = 'primer_proyecto_nucleo/Debug/primer_proyecto_nucleo.elf'
elf = 'primer_proyecto_nucleo/Debug/primer_proyecto_nucleo.elf'
```

Con la mejora de un circuito por proyecto, va adentro de la carpeta del proyecto y la ruta se acorta:

```
[wokwi]
version = 1
firmware = 'Debug/tp2.elf'
elf = 'Debug/tp2.elf'
```

De dónde sale esto. El método es el de la cátedra: las clases «STM32CubeIDE — Instalación y primer proyecto» y «De CubeIDE a Wokwi» (semana 4), el TP2 (semana 5) y el TP3 (semana 6). Las cajas verde azulado son agregados. «Compilar desde VS Code» se probó el 29/09/2026 sobre el proyecto de la clase: compila por consola en una ruta con espacios, un error de sintaxis sale con archivo y línea, y el **.elf** queda donde lo busca **wokwi.toml**. Los comandos de Wokwi que se nombran son los que declara la extensión 3.7.0 instalada.